Laboratorio de Seguridad — Spring Boot (Java)

Aseguramiento del aplicativo de la tienda · Desarrollo de Aplicaciones Web

Proyecto del examen final: Tienda en Línea Modular

Integrantes del equipo	Gustavo Andrés Núñez Vega Cristian Camilo Chaparro
Fecha	27/10/2026
Repositorio	github.com/CristianCamiloChaparro/tienda-modular-backend — rama `seguridad`
Contenido	Diagnóstico inicial, hallazgos y correcciones (Partes 0 a 6), auditoría OWASP Top 10, matriz de riesgos, plan ante un incidente, datos personales y preguntas finales

> **Proyecto:** API REST en Spring Boot 3.5 + Spring Security 6 + PostgreSQL, con un frontend React (Vite) en `frontend/`. Los nombres de la guía se adaptaron al proyecto: `Usuario`, `Pedido`, `/api/v1/productos`, `/api/v1/pedidos`; en lugar de MongoDB Atlas se usa PostgreSQL (Render en producción). La evidencia completa (salidas de `curl`, `psql`, `openssl` y Chromium) está en `docs/laboratorio/`.

Commits del laboratorio (uno por parte):

Parte	Commit
0 — Diagnóstico	`72f9051` Parte 0: informe de seguridad y diagnóstico inicial
1 — HTTPS	`81f5c98` Parte 1: HTTPS local con la clave del certificado fuera del código
2 — Login	`084ac3c` Parte 2: hash, sesión y límite de intentos de login
3 — Roles	`41e7c75` Parte 3: roles y control de acceso
4 — XSS	`2b81cc4` Parte 4: XSS demostrado, corregido y reforzado con CSP
5 — CSRF	`11eeccb` Parte 5: CSRF con token de doble envío y cookies SameSite=Strict
6 — Secretos	`30341b6` Parte 6: secretos, errores genéricos y dependencias
7 — Auditoría	Parte 7: auditoría OWASP, pruebas de diseño inseguro e informe (último commit de la rama)

Resultado de las pruebas automáticas al cierre: backend `./mvnw test` → **73 pruebas, 0 fallos** (24 nuevas de seguridad); frontend `npm test` → **51 pruebas, 0 fallos** (1 nueva de XSS); suite end-to-end `node e2e-tests/run-all.js` → **42/42**.

Parte 0 — Preparación y diagnóstico

1. El proyecto arranca y la suite de pruebas pasa **antes** de tocar nada (`./mvnw test`).
2. Se trabajó en una rama aparte (`seguridad`) para poder volver atrás.
3. Se creó este archivo `INFORME_SEGURIDAD.md`.

Diagnóstico inicial (estado del proyecto antes del laboratorio)

Pregunta	Qué se buscó	Sí / No	Detalle encontrado
⚠ ¿Hay contraseñas guardadas tal cual?	Campo `password` que se guarda sin hash	**No**	`Usuario.passwordHash` guarda un hash BCrypt (`AuthService.registrar`).
⚠ ¿Hay credenciales en el código?	`password`, `secret` en `application.properties` o en clases Java	**Sí**	Valores por defecto en la configuración: `spring.datasource.password=${DB_PASSWORD:postgres}`, `server.ssl.key-store-password=${SSL_KEYSTORE_PASSWORD:tienda-local}`, `-storepass tienda-local` en `scripts/generar-certificado.sh` y `DB_PASSWORD:-tienda_local` en `docker-compose.yml`.
⚠ ¿Hay archivos de secretos en git?	`git ls-files` muestra `.env` o similares	**No** (archivos) / **Sí** (historial)	`git ls-files` no muestra `.env`, `.p12` ni `.pem`. Pero `git log -p -S"Admin12345"` muestra la clave del administrador escrita en el commit `a902452` (quitada en `dfc48de`).
⚠ ¿Se muestra texto de usuarios como HTML?	`innerHTML`, `dangerouslySetInnerHTML`	**No**	El frontend es React y no usa `dangerouslySetInnerHTML` ni `innerHTML`; el backend además limpia el HTML con jsoup.
⚠ ¿Se puede ver un recurso de otro usuario cambiando el id?	Rutas `/api/pedidos/:id` sin comprobar propietario	**No** (pedidos) / **Sí** (carritos)	`GET /api/v1/pedidos/{id}` responde 404 si el pedido es de otro cliente. Los carritos son anónimos: quien conozca el id (UUID) de un carrito puede verlo y modificarlo.
⚠ ¿Las rutas de administrador solo están «ocultas» en el frontend?	Botones escondidos, endpoint sin rol	**No**	`SecurityConfig` exige `hasRole("ADMIN")` en POST/PUT/DELETE de categorías y productos, en `GET /api/v1/pedidos` y en `PATCH /api/v1/pedidos/*/estado`.
⚠ ¿CORS está abierto a cualquier origen?	`@CrossOrigin` sin parámetros, `allowedOrigins("*")`	**No**	`CorsConfig` solo permite los orígenes de `CORS_ORIGINS` (por defecto `http://localhost:5173`).
⚠ ¿El proyecto usa HTTP simple?	Se abre con `http://`	**Sí** (desarrollo)	En producción (Render) y con Docker Compose (Caddy) hay HTTPS, pero el arranque local por defecto es `http://localhost:8080` y el perfil `https` usaba una clave escrita en el código.

Comandos usados:

```
git ls-files | grep -Ei '\.env|\.p12|\.pem|\.key|secret'      # sin resultados
git log --all -p -S"Admin12345" --format='%h %ad %s' --date=short
grep -rn "innerHTML\|dangerouslySetInnerHTML" frontend/src     # sin resultados
grep -rn "@CrossOrigin\|allowedOrigins" src/main               # solo CorsConfig con lista de orígenes
```

Tabla de hallazgos

#	Hallazgo	OWASP	Evidencia (cómo lo comprobé)	Corrección aplicada	Estado
1	Credenciales por defecto escritas en la configuración (BD `postgres`, keystore `tienda-local`, Compose `tienda_local`)	A02 / A04	Diagnóstico: búsqueda de `password` en `application*.properties`, el script y `docker-compose.yml`	Sin valores por defecto: `DB_PASSWORD` y `SSL_PASSWORD` vienen del `.env` o del entorno; `VerificadorDeSecretos` detiene el arranque con un mensaje claro si faltan	**Corregido** (Partes 1 y 6)
2	Clave del administrador `Admin12345` en el historial de git	A04 / A07	`git log --all -p -S"Admin12345"` → commits `a902452` (la agrega) y `dfc48de` (la quita)	Ya no está en el código. Como el historial no se borra, la clave se considera filtrada: hay que **rotarla** en producción (ver Plan ante un incidente)	**Mitigado** — falta rotar la clave en Render
3	HTTP simple en desarrollo y clave del certificado escrita en el código	A04	Arranque local en `http://localhost:8080`; `-storepass tienda-local` en el script	Perfil `https`: `https://localhost:8443`, el 8080 redirige (302); certificado con mkcert o keytool y clave en `SSL_PASSWORD`; HSTS de 5 min en local y 1 año en producción	**Corregido** (Parte 1)
4	Sin límite de intentos de inicio de sesión (fuerza bruta)	A07	6 intentos seguidos con clave incorrecta: todos respondían 401	`LimitadorLogin`: 5 fallos por IP cada 15 min; el sexto responde **429** con `Retry-After: 900`	**Corregido** (Parte 2)
5	Contraseña mínima de 8 caracteres y BCrypt de costo 10	A07 / A04	`RegistroRequest` (`@Size(min = 8)`) y `new BCryptPasswordEncoder()`	Mínimo 10 caracteres (servidor y formulario) y BCrypt costo 12 (`$2a$12$…` en la BD)	**Corregido** (Parte 2)
6	Sesión de 2 horas y cookies `SameSite=Lax`	A07	`server.servlet.session.timeout=2h`; cabecera `Set-Cookie`	Sesión de 15 min sin actividad; `JSESSIONID HttpOnly; Secure; SameSite=Strict`; `XSRF-TOKEN Secure; SameSite=Strict`	**Corregido** (Partes 2 y 5)
7	Los intentos de login fallidos no se registraban	A09	Log del servidor sin rastro de los 401	`WARN Inicio de sesion fallido para '<correo>' desde <IP>` y `WARN … bloqueado por exceso de intentos` (con limpieza de CR/LF para evitar inyección en el log)	**Corregido** (Parte 2)
8	Consulta de pedido sin el propietario en el filtro (comprobación posterior en Java)	A01	Revisión de `PedidoService.obtener`: buscaba por id y luego comparaba	`findWithItemsByIdAndUsuarioId`: la consulta incluye siempre al propietario. B pidiendo el pedido de A → **404**	**Corregido** (Parte 3; ya respondía 404, se reforzó)
9	Carritos anónimos accesibles por su id	A01	`GET /api/v1/carritos/{uuid}` desde otra cuenta → 200	El id es un UUID v4 (122 bits aleatorios): no se puede adivinar ni recorrer. El carrito no guarda datos personales y el pago exige sesión	**Aceptado** (ver matriz de riesgos)

#	Hallazgo	OWASP	Evidencia (cómo lo comprobé)	Corrección aplicada	Estado
10	XSS almacenado en campos de texto libre	A05	Payload `<img src=x onerror="alert('XSS')">` en nombre de usuario, categoría y producto	No vulnerable: jsoup elimina el HTML antes de guardar, React escapa la salida y la CSP bloquea scripts en línea (comprobado en Chromium)	**Verificado** (Parte 4)
11	CSRF en operaciones que modifican datos	A01	Página `ataque-csrf.html` en otro sitio con la sesión del admin abierta	No vulnerable: token de doble envío (403 sin cabecera), `SameSite=Strict` y CORS con orígenes permitidos	**Verificado** (Parte 5)
12	Errores no controlados sin manejador final y `IllegalArgumentException` devolviendo su mensaje interno	A10 / A02	Revisión de `GlobalExceptionHandler`	Manejador final: 500 genérico hacia afuera y detalle en el log; errores de Spring MVC (404/405) conservan su código; `server.error.include-*=never`	**Corregido** (Parte 6)
13	Dependencias desactualizadas y sin auditoría automática	A03	`versions:display-dependency-updates` y `npm audit` (2 vulnerabilidades **altas** en vite/launch-editor)	Spring Boot 3.5.6 → 3.5.16 (Tomcat 10.1.55, Spring Security 6.5.11), jsoup 1.23.2, springdoc 2.8.17; `npm audit fix` → 0 vulnerabilidades; Dependabot semanal	**Corregido** — dependency-check de OWASP pendiente (NVD bloqueado en la red del laboratorio)
14	Cantidades negativas, precio manipulado o rol en el cuerpo de la petición	A06 / A08	`cantidad: -5`, `precio: 1`, `"rol": "admin"`	No vulnerable: `@Min(1)` → 400; el precio y el total se leen de la BD; el DTO de registro no tiene campo `rol`	**Verificado** (Parte 7)

Parte 1 — HTTPS local

- `application-https.properties`: `server.port=8443`, keystore PKCS12 en `certs/tienda-local.p12` (fuera del classpath y en `.gitignore`) y `server.ssl.key-store-password=${SSL_PASSWORD}`.
- `spring.config.import=optional:file:.env[.properties]` lee el `.env` de la raíz del proyecto.
- `scripts/generar-certificado.sh`: con **mkcert** instalado emite un certificado de la CA local (candado sin advertencia; mkcert usa la clave `changeit`); si no, genera uno **autofirmado** con keytool (el navegador muestra la advertencia esperada).
- Reto opcional hecho: un segundo conector de Tomcat en el 8080 redirige a HTTPS (`HttpsRedirectConfig`).
- HSTS: `max-age=300` en localhost (como pide la guía) y `31536000` en producción; también se añadió en nginx (`HSTS_MAX_AGE`).

Evidencia (`parte1-https.txt`):

```
$ curl -sI http://localhost:8080/api/v1/productos
HTTP/1.1 302
Location: https://localhost:8443/api/v1/productos
$ curl -skI https://localhost:8443/api/v1/productos
HTTP/1.1 200
Strict-Transport-Security: max-age=300 ; includeSubDomains
$ openssl s_client -connect localhost:8443 | openssl x509 -noout -subject -issuer -dates
subject=C = CO, O = Proyecto de Aula, OU = Tienda Modular, CN = localhost
notBefore=Oct 27 22:18:12 2026 GMT  notAfter=Oct 27 22:18:12 2027 GMT
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
```

Pregunta: ¿qué garantías da HTTPS que HTTP no da? ¿Garantiza que el sitio sea honesto? HTTPS (TLS) da tres garantías: **confidencialidad** (el contenido viaja cifrado; quien esté en la misma red Wi-Fi no puede leer contraseñas, cookies de sesión ni datos de tarjeta), **integridad** (nadie puede modificar la respuesta en el camino, por ejemplo inyectando un script) y **autenticidad del servidor** (el certificado prueba que hablamos con el dueño del dominio, no con un impostor en el medio). **No garantiza que el sitio sea honesto:** el certificado solo dice que el dominio es de quien lo opera. Una página de phishing como `tienda-pagos-seguro.com` puede tener un candado perfectamente válido. HTTPS protege el canal, no la intención de quien está del otro lado ni la seguridad de su código.

Parte 2 — Registro y login: hash, token y límite de intentos

Decisión de diseño (token). La guía usa un JWT en una cookie `HttpOnly`. El proyecto ya usaba una **sesión de servidor**: la cookie `JSESSIONID` es un **token opaco y aleatorio** que el servidor asocia a la sesión. Se mantuvo porque cumple los mismos objetivos de la guía (token con vencimiento, guardado en una cookie que JavaScript no puede leer, con `HttpOnly`, `Secure` y `SameSite=Strict`). Además, a diferencia de un JWT, se puede **revocar** al instante (logout o expiración en el servidor), y no cambia el frontend, las pruebas ni el despliegue. Se ajustó el vencimiento a **15 minutos sin actividad**, igual que el token de la guía, y se cambia el id de sesión al autenticarse (fijación de sesión).

Requisito de la guía	Implementación en el proyecto
Hash lento con sal	`BCryptPasswordEncoder(12)` en `AuthConfig`
Correo y contraseña validados en el servidor (mínimo 10)	`RegistroRequest`: `@Email`, `@Size(min = 10, max = 72)`, letra y número
El rol no viene del cuerpo	`RegistroRequest` no tiene campo `rol`; `AuthService` asigna `Rol.CLIENTE`
Token firmado con vencimiento en cookie HttpOnly	Sesión de servidor de 15 min; `JSESSIONID` `HttpOnly; Secure; SameSite=Strict`
Mensaje genérico	«Correo o contraseña incorrectos» en ambos casos
Límite de intentos	`LimitadorLogin` (5 fallos por IP / 15 min) → **429** + `Retry-After`

Evidencia (`parte2-autenticacion.txt`):

```
POST /auth/registro {... "rol":"ADMIN"}           -> 201 {"rol":"CLIENTE"}
POST /auth/registro con clave de 9 caracteres      -> 400 "La contrasena debe tener entre 10 y 72
caracteres"
psql: laura.lab@correo.com | CLIENTE | $2a$12$gG12E5F9jtwz9H7PST90seVZP4qzvYr8qZneT4Dk7l6EbqQoKJGmK
POST /auth/login correcto -> 200
  Set-Cookie: JSESSIONID=<valor>; Path=/; Secure; HttpOnly; SameSite=Strict
GET /pedidos/mios sin sesión -> 401
Intentos 1-5 con clave incorrecta -> 401 ; intento 6 -> 429 (Retry-After: 900)
document.cookie en el navegador -> 'XSRF-TOKEN=<valor>'   (JSESSIONID no aparece: HttpOnly funciona)
WARN Inicio de sesion fallido para 'laura.lab@correo.com' desde 127.0.0.1
WARN Inicio de sesion bloqueado por exceso de intentos desde 127.0.0.1
```

Parte 3 — Roles y control de acceso

Rol	Ver catálogo	Comprar	Crear/editar/borrar productos	Ver todos los pedidos
Visitante	Sí (`GET` público)	No (401)	No (401)	No (401)
Cliente	Sí	Sí (`POST /pedidos` autenticado)	No (**403**)	Solo los suyos (`/pedidos/mios`, `/pedidos/{id}` propio)

Rol	Ver catálogo	Comprar	Crear/editar/borrar productos	Ver todos los pedidos
Administrador	Sí	Sí	Sí (`hasRole("ADMIN")`)	Sí (`GET /pedidos`)

Las reglas viven en `SecurityConfig` (servidor), no en el frontend. Los roles se guardan como `CLIENTE` y `ADMIN` y Spring Security les antepone `ROLE_`. El primer administrador lo crea `AdminSeeder` con `ADMIN_PASSWORD`; no hay ningún endpoint público para crear administradores.

IDOR: con dos clientes (A y B), A crea un pedido; B pide `GET /api/v1/pedidos/<id de A>` → **404**, la misma respuesta que para un id inexistente, así que ni siquiera se revela que el pedido existe. La consulta incluye siempre al propietario (`findWithItemsByIdAndUsuarioId`).

Evidencia (`parte3-control-de-acceso.txt`):

```
(cliente B) GET /api/v1/pedidos/1  (pedido de A)  -> 404 "Pedido con id 1 no encontrado"
(cliente A) GET /api/v1/pedidos/1                 -> 200
(cliente B) DELETE /api/v1/productos/7            -> 403 "No tienes permisos para realizar esta operacion"
(cliente B) GET /api/v1/pedidos                   -> 403
(administrador) DELETE /api/v1/productos/7        -> 204
```

Parte 4 — XSS: demostrar, corregir y reforzar

4.1 Demostración. Los textos escritos por usuarios que otros ven son: nombre del usuario (cabecera), nombre y descripción de categorías y productos, y dirección de envío (vista del administrador). Se publicó `<img src=x onerror="alert('XSS')">` en todos ellos. **No apareció la alerta**: el proyecto ya tenía tres capas de defensa.

Capa	Medida	Resultado
Entrada (servidor)	`TextoSeguro` (jsoup, lista blanca vacía, repetido hasta que el texto no cambie)	`<img …>Mario` se guarda como `Mario`; solo HTML → 400 «no puede quedar vacío»
Entrada (servidor)	`imagenUrl` solo acepta `http(s)://`	`javascript:alert(1)` → 400
Salida (frontend)	React escapa todo; no se usa `dangerouslySetInnerHTML`	Prueba `ProductCard.test.tsx`: el payload se muestra como texto y no se crea ningún `<img>`
Navegador	`Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none'…` + `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`	Bloquea scripts y manejadores en línea aunque llegaran a la página

4.3 Prueba de defensa en profundidad (`node docs/laboratorio/demo-csp.mjs`, Chromium real). Se ejecutó el código vulnerable de la guía (`tarjeta.innerHTML = '<p>' + texto + '</p>'`):

```
innerHTML vulnerable SIN Content-Security-Policy   -> Alertas ejecutadas: XSS
innerHTML vulnerable CON la CSP del servidor        -> Alertas ejecutadas: ninguna
  Consola: Executing inline event handler violates the following Content Security Policy
           directive 'script-src 'self''. ... The action has been blocked.
Corrección con textContent -> <p>&lt;img src=x onerror="alert('XSS')"&gt;</p>  (0 imágenes creadas)
```

Evidencia completa: `parte4-xss.txt`.

Parte 5 — CSRF

Servidor. CSRF de Spring Security activo con el patrón de **doble envío de cookie**: el servidor entrega `XSRF-TOKEN` (legible por JavaScript del mismo origen) y exige la cabecera `X-XSRF-TOKEN` en todo `POST`, `PUT`, `PATCH` y `DELETE`. A diferencia de la guía, **el login y el registro también están protegidos** (evita el *login CSRF*, en el que un atacante hace que la víctima inicie sesión en la cuenta del atacante). El token se renueva al iniciar y cerrar sesión. **Frontend.** `frontend/src/services/http.ts` lee la cookie y envía la cabecera en todas las llamadas que modifican datos; si recibe 403 por CSRF, pide un token nuevo y reintenta una vez.

Evidencia (`parte5-csrf.txt`, `node docs/laboratorio/demo-csrf.mjs` en Chromium):

```
Sesión del administrador abierta: POST /auth/login -> 200
   cookie JSESSIONID: HttpOnly=true Secure=true SameSite=Strict
   cookie XSRF-TOKEN: HttpOnly=false Secure=true SameSite=Strict
Petición legítima desde el mismo origen (X-XSRF-TOKEN enviada): POST /categorias -> 201
Página maliciosa en https://atacante.local/premio.html (formulario POST a la API)
   Cookies que el navegador adjuntó: (ninguna)
   Respuesta del servidor: 403
¿Se creó "categoria-del-atacante"? NO
curl con la cookie de sesión del admin pero SIN cabecera X-XSRF-TOKEN -> 403 "Token CSRF ausente o
invalido"
curl con X-XSRF-TOKEN inventado                                    -> 403
curl con X-XSRF-TOKEN copiado de la cookie                         -> 201
```

Pregunta: ¿qué pasaría si alguien escribiera `.csrf(csrf -> csrf.disable())` «porque da error 403»? ¿Cuándo sí se justifica desactivarlo? El 403 no es un error: es la protección funcionando. Al desactivarla, cualquier página que visite el administrador con la sesión abierta podría crear, editar o borrar productos y cambiar estados de pedidos en su nombre, porque el navegador adjunta la cookie de sesión automáticamente. La solución correcta es enviar la cabecera `X-XSRF-TOKEN` desde el frontend. Desactivarlo solo se justifica en una API **que no usa cookies**, es decir, que autentica únicamente con la cabecera `Authorization: Bearer <token>` (clientes móviles, servicio a servicio). El navegador nunca añade esa cabecera por su cuenta, así que un sitio ajeno no puede falsificar la petición. No es el caso de este proyecto.

Pregunta: si ya usan `SameSite=Strict`, ¿por qué se añade además el token? Por **defensa en profundidad**: (1) **navegadores antiguos** o configuraciones que no aplican `SameSite` siguen enviando la cookie; (2) `SameSite` mira el **sitio** (dominio registrable), no el origen: un **subdominio** comprometido o vulnerable a XSS (`blog.tienda.com`) es «el mismo sitio» que `tienda.com` y sus peticiones sí llevarían la cookie. El token, en cambio, exige leer la cookie desde el **mismo origen**; (3) si en el futuro alguien cambia `Strict` por `Lax` o `None` (por ejemplo, para un retorno desde una pasarela de pagos), el token sigue protegiendo; (4) el token también protege peticiones `GET` de nivel superior que `Lax` permite. Ninguna medida sola alcanza.

Parte 6 — Secretos, CORS, errores y dependencias

6.1 Secretos fuera del código. Todo secreto va en `.env` (en `.gitignore`, junto con `certs/`, `*.p12`, `*.pem` y `*.key`). `.env.example` documenta las variables **sin valores reales**. Ya no hay claves por defecto: `spring.datasource.password=${DB_PASSWORD}` y `server.ssl.key-store-password=${SSL_PASSWORD}`; `docker-compose.yml` usa `${DB_PASSWORD:?…}`. Spring Boot, al no encontrar la variable, usaría el texto literal `${DB_PASSWORD}` como clave y fallaría con un error confuso de conexión, así que `VerificadorDeSecretos` detiene el arranque con un mensaje claro:

```
$ java -jar target/tienda-modular-backend-1.0.0.jar      (sin .env)
java.lang.IllegalStateException: Falta el secreto DB_PASSWORD: definalo en el archivo .env junto al
pom.xml (ver .env.example) o como variable de entorno
```

`git log --all -p -S"Admin12345"` muestra que la clave inicial del administrador **sí se subió** al historial (commit `a902452`) y se quitó en `dfc48de`. Como en el caso de Uber 2016, borrarla del código no basta: **el historial ya la filtró y hay que rotarla** (hallazgo 2).

6.2 Configuración de la base de datos (equivalente a Atlas en Render PostgreSQL):

Revisión	Situación	Acción
Network Access (equivalente a `0.0.0.0/0`)	El backend usa la **URL interna** de Render (red privada). La URL externa de la base de datos acepta por defecto conexiones desde cualquier IP (`0.0.0.0/0`); no se pudo revisar el panel desde el laboratorio	**Pendiente:** en Render → *tienda-db* → *Networking / Access Control*, quitar `0.0.0.0/0` y dejar solo la IP del equipo cuando se necesite conectarse desde afuera
Database Access (permiso mínimo)	Render crea un único usuario dueño de la base `tienda_modular`, sin permisos de superusuario sobre el servidor	Aceptable. Mejora: usuario de la aplicación solo con DML y migraciones con Flyway (`ddl-auto=validate`)

6.3 CORS. El frontend y la API se sirven en el **mismo origen** (nginx o Vite reenvían `/api`), así que no hace falta CORS. Solo quedan permitidos los orígenes de `CORS_ORIGINS` (nunca `*`), con `allowCredentials(true)` y las cabeceras `Content-Type`, `Accept` y `X-XSRF-TOKEN`. No hay ningún `@CrossOrigin`. En la prueba de CSRF, la petición desde `https://atacante.local` fue rechazada además por CORS.

6.4 Errores genéricos. `server.error.include-message/stacktrace/binding-errors=never`. `GlobalExceptionHandler` tiene un manejador final que responde 500 con «Ocurrió un error interno» y escribe el detalle en el log; los errores de Spring MVC (404, 405…) conservan su código y una `IllegalArgumentException` ya no devuelve su mensaje interno (`parte6-secretos-errores.txt`):

```
GET /api/v1/pedidos/abc                -> 400 "El parametro 'id' tiene un valor invalido"
POST ... con JSON mal formado          -> 400 "El cuerpo JSON esta mal formado o tiene tipos incorrectos"
DELETE /api/v1/auth/me                 -> 405 "Method Not Allowed"
Error no controlado (prueba unitaria)  -> 500 "Ocurrio un error interno. Intente de nuevo mas tarde" (sin
jdbc, clases ni trazas)
```

6.5 Dependencias (`parte6-dependencias.txt`):

Herramienta	Resultado
`./mvnw dependency:tree`	10 dependencias directas, 112 líneas en total
`./mvnw versions:display-dependency-updates`	Spring Boot 3.5.6 → **3.5.16**, springdoc 2.8.9 → **2.8.17**, jsoup 1.18.3 → **1.23.2** (aplicadas; con ellas: Tomcat 10.1.46 → 10.1.55, Spring Security 6.5.5 → 6.5.11, Jackson 2.19.2 → 2.21.4, Logback 1.5.18 → 1.5.34, driver PostgreSQL 42.7.7 → 42.7.11)
`./mvnw org.owasp:dependency-check-maven:check`	**No se pudo completar en el laboratorio:** la red bloqueó la descarga de la base NVD (403). Pendiente correrlo en casa con `-DnvdApiKey=<clave>`
`npm audit` (frontend)	**2 vulnerabilidades altas** (vite 8.3.0–8.3.2 y launch-editor; solo herramientas de desarrollo) → `npm audit fix` → **0 vulnerabilidades**
Dependabot	`.github/dependabot.yml`: revisión semanal de Maven, npm y Docker

Vulnerabilidades de nivel alto o crítico encontradas: **2 altas** (npm, ambas corregidas). En Maven no se pudo medir con dependency-check; se mitigó actualizando a las últimas versiones de parche de la línea 3.5.

Parte 7 — Auditoría OWASP Top 10 (2025)

OWASP	Pregunta de revisión	Resultado en el proyecto	Dónde se comprobó
A01 Control de acceso roto	¿Cada ruta verifica rol **y** propietario en el servidor?	Sí. Reglas por rol en `SecurityConfig`; pedidos filtrados por propietario (404); `anyRequest().denyAll()` para lo no declarado. Riesgo aceptado: carritos anónimos por UUID	Parte 3
A02 Configuración insegura	¿CORS, DEBUG, BD y cabeceras están bien?	Sí. CORS con lista de orígenes, `show-sql=false`, errores sin traza, CSP/HSTS/nosniff/X-Frame-Options/Referrer-Policy. Pendiente: acceso externo a la BD en Render	Partes 4 y 6
A03 Cadena de suministro	¿Se auditaron las dependencias?	Sí, parcialmente: `npm audit` (2 altas → 0), Maven actualizado y Dependabot activo. Pendiente: dependency-check con API key de NVD	Parte 6.5
A04 Fallas criptográficas	¿HTTPS activo? ¿Contraseñas con hash lento? ¿Secretos fuera del código?	Sí. TLS 1.3, HSTS, BCrypt costo 12, secretos en `.env` o en el entorno. Hallazgo: clave del admin en el historial (rotar)	Partes 1, 2 y 6
A05 Inyección (incluye XSS)	¿Salida escapada y entradas validadas por tipo?	Sí. JPA con consultas parametrizadas (sin SQL concatenado), Bean Validation en los DTO, jsoup + React + CSP contra XSS	Partes 2 y 4
A06 Diseño inseguro	¿Hay lógica que se pueda abusar (cantidades negativas, cupones sin límite)?	No se encontró. `cantidad: -5` → 400; precio y total siempre desde la BD; stock con bloqueo de fila (`PESSIMISTIC_WRITE`); no hay cupones	Parte 7.1
A07 Fallas de autenticación	¿Límite de intentos, mensajes genéricos, tokens con vencimiento?	Sí. 5 intentos / 15 min → 429, mensaje genérico, sesión de 15 min, id de sesión nuevo al iniciar sesión	Parte 2
A08 Integridad de software o datos	¿El token está firmado y se verifica? ¿Se confía en datos del cliente (precio, rol)?	Sí. La sesión es un identificador aleatorio que solo valida el servidor (no lleva datos que se puedan alterar); el rol y el precio nunca se toman del cliente	Partes 2 y 3
A09 Registros y alertas	¿Se registran los intentos de login fallidos y los errores?	Sí (WARN con IP, ERROR con traza). Pendiente: alertas y registros centralizados	Parte 6.4
A10 Condiciones excepcionales	¿Los errores fallan de forma segura (niegan, no permiten)?	Sí. Un token CSRF inválido o una sesión vencida niegan (403/401); una excepción no controlada responde 500 genérico sin ejecutar la operación (las transacciones hacen rollback); sin `DB_PASSWORD` el servidor no arranca	Parte 6.4

7.1 Dos pruebas de diseño inseguro (A06)

Evidencia (`parte7-diseno-inseguro.txt`), producto 4 con precio real 52.000:

```
POST /carritos/{id}/items {"productoId":4,"cantidad":-5}                    -> 400 "La cantidad debe ser
al menos 1"
POST /carritos/{id}/items {"productoId":4,"cantidad":2,"precio":1,"total":1}  -> 200 precioUnitario
52000.00, total 104000.00
POST /auth/registro {..., "rol":"admin"}                                    -> 201 {"rol":"CLIENTE"}
```

El precio y el total enviados se ignoran: el servidor los lee de la base de datos y vuelve a calcular el total al pagar. El campo `rol` no existe en el DTO.

Pruebas automáticas que respaldan el informe

`SeguridadIntegrationTest` (15 pruebas sobre la aplicación completa), `LimitadorLoginTest`, `GlobalExceptionHandlerTest`, `VerificadorDeSecretosTest` y `ProductCard.test.tsx` (XSS en el frontend), además de las pruebas previas (`FlujoCompraIntegrationTest`, `TextoSeguroTest`, CSRF en `CategoriaControllerTest`).

7.2.1 Matriz de riesgos (riesgos que siguen presentes)

#	Riesgo	Probabilidad	Impacto	Tratamiento
1	La clave inicial del administrador (`Admin12345`) quedó en el historial público de git. Si el administrador de producción se creó con ella, alguien podría entrar como administrador	Media	Alto	**Mitigar ya:** cambiar la contraseña del administrador en producción (actualizar su hash en la tabla `usuarios`) y definir un `ADMIN_PASSWORD` nuevo en Render
2	El límite de intentos está en memoria y por IP: se reinicia con el servidor, no se comparte entre instancias, no frena ataques desde muchas IP y puede bloquear a usuarios que comparten IP (NAT)	Media	Medio	**Mitigar:** Bucket4j con almacenamiento compartido (Redis) o API Gateway, límite también por cuenta y CAPTCHA tras varios fallos
3	La cuenta de administrador solo tiene contraseña (sin segundo factor)	Baja	Alto	**Mitigar:** MFA (TOTP) para el rol ADMIN
4	Los registros solo quedan en la consola de Render: no hay alertas ni retención	Media	Medio	**Mitigar:** enviar los registros a un servicio centralizado con alertas (por ejemplo, muchos 401/429 seguidos o errores 500)
5	Carritos anónimos accesibles por quien conozca su UUID	Baja	Bajo	**Aceptar:** el UUID v4 no se puede adivinar, el carrito no contiene datos personales y pagar exige sesión. Mejora futura: asociar el carrito a la sesión del usuario

Riesgo residual adicional: la auditoría de Maven con OWASP dependency-check no se completó (NVD bloqueado). Tratamiento: correrla en casa y mantener Dependabot activo.

7.2.2 Plan ante un incidente: se filtró la cadena de conexión de la base de datos

Adaptado a PostgreSQL en Render (equivale a la cadena de Atlas), siguiendo el ciclo de NIST SP 800-61:

1. **Preparación.** Tener a mano quién responde y cómo contactarlo; secretos solo en variables de entorno de Render y en `.env` (nunca en git); Dependabot y escaneo de secretos de GitHub activos; respaldos de la base de datos y un procedimiento probado para cambiar credenciales.
2. **Detección y análisis.** Confirmar dónde apareció (commit, captura, chat), desde cuándo y si es la URL interna o la externa. Revisar en Render las conexiones y la actividad de la base de datos, y en los registros del backend si hay accesos o cambios anómalos (pedidos o productos modificados, usuarios ADMIN nuevos). Clasificar la gravedad y qué datos pudieron verse (datos personales → Ley 1581).
3. **Contención.** De inmediato: **rotar la contraseña** del usuario de la base de datos en Render, restringir el acceso externo (quitar `0.0.0.0/0`), actualizar `DB_PASSWORD` en el backend y redesplegar. Invalidar las sesiones activas (reiniciar el backend) y, si hubo acceso, cambiar también `ADMIN_PASSWORD`.
4. **Erradicación.** Quitar la credencial del código y limpiar el historial (`git filter-repo` o BFG, y luego `push --force` coordinado con el equipo); eliminar cualquier usuario o dato que haya creado el atacante y corregir la causa (por ejemplo, un `.env` que no estaba en `.gitignore`).
5. **Recuperación.** Restaurar datos desde un respaldo si hubo alteraciones, verificar que la aplicación funciona con la nueva credencial, y vigilar de cerca los registros y conexiones durante los días siguientes.

6. **Lecciones aprendidas.** Postmortem sin culpables: qué falló y qué control lo habría evitado (escaneo de secretos antes del commit, revisión de PR, alertas). Actualizar este plan. Si se expusieron datos personales, notificar a la Superintendencia de Industria y Comercio y a los titulares afectados, como exige la Ley 1581 de 2012.

7.2.3 Datos personales (Ley 1581 de 2012)

Dato	Dónde	Para qué	¿Hace falta?
Nombre	`usuarios.nombre`	Saludar al cliente y emitir el pedido	Sí
Correo	`usuarios.email`	Identificar la cuenta e iniciar sesión	Sí
Hash de la contraseña	`usuarios.password_hash`	Autenticación (nunca la contraseña)	Sí
Dirección de envío y teléfono	`pedidos.direccion_envio`, `pedidos.telefono`	Entregar el pedido y contactar al cliente	Sí, pero solo mientras se gestiona la entrega y la garantía
Historial de compras	`pedidos`, `items_pedido`	Historial del cliente y contabilidad	Sí (obligación contable)
Marca y últimos 4 dígitos de la tarjeta, referencia de pago	`pedidos.medio_pago`, `referencia_pago`	Conciliar el pago y atender reclamos	Sí. **No** se guardan el número completo, el CVV ni el vencimiento
Correo e IP en los registros de inicio de sesión fallido; correo en el log de la pasarela	Log del servidor	Detectar ataques (A09)	Sí, con un tiempo de retención limitado

Qué sobra o se debería ajustar: (1) la dirección y el teléfono se pueden **anonimizar** después del plazo de garantía en lugar de guardarlos para siempre; (2) el log de la pasarela simulada escribe el correo del cliente: basta con el id del pedido; (3) los registros con correo e IP necesitan una **retención limitada** (por ejemplo, 90 días). **Qué falta para cumplir la ley:** según los principios de finalidad, libertad, transparencia y seguridad de la Ley 1581, falta una **política de tratamiento de datos** y una **autorización expresa** del titular en el registro (casilla de aceptación). También faltan mecanismos para ejercer los derechos de **conocer, actualizar, rectificar y suprimir** (no hay opción para editar los datos ni eliminar la cuenta). Hoy se cumple el principio de **seguridad** (hash, HTTPS, control de acceso) y el de **acceso restringido** (cada cliente solo ve sus pedidos).

Cierre

1. **¿Cuál fue el hallazgo más grave de su proyecto y por qué?** La **clave del administrador en el historial de git** (`Admin12345`, hallazgo 2). El repositorio está en GitHub y Render lo despliega como repositorio público, así que cualquiera que lea el historial puede verla. Si el administrador de producción se creó con ella, da control total: crear, editar y borrar productos y ver todos los pedidos con datos personales. Antes del laboratorio tampoco había límite de intentos de inicio de sesión, lo que facilitaba probarla. Quitarla del código no basta; hay que rotarla.

2. **¿Qué defensa creen que habría evitado el caso de la clase que más los impactó? Uber (2016):** las credenciales de AWS estaban en un repositorio de GitHub y desde ahí se robaron los datos de 57 millones de usuarios. Lo habrían evitado los **secretos fuera del código** (variables de entorno o `.env` ignorado por git, como en la Parte 6), el **escaneo de secretos** antes de cada commit, la **rotación inmediata** tras la filtración y el **permiso mínimo** de esas credenciales. Es justo el mismo tipo de error que encontramos en

nuestro historial. (Equifax se habría evitado aplicando a tiempo el parche de Apache Struts, Parte 6.5. British Airways, con una CSP estricta e integridad de subrecursos que impidieran cargar el script modificado.)

3. **¿Qué dejarían como mejora futura? MFA** para la cuenta de administrador; límite de intentos **distribuido** (Bucket4j + Redis) y también por cuenta; **registros centralizados con alertas**; **pruebas automáticas de seguridad en CI** (OWASP dependency-check con API key, `npm audit` y un escaneo dinámico con OWASP ZAP en cada PR); escaneo de secretos con *push protection* en GitHub; migraciones con Flyway y un usuario de base de datos con permisos mínimos; y la política de tratamiento de datos con la autorización del titular.

> **Recuerden:** ninguna medida alcanza sola. La seguridad es **defensa en profundidad** y se piensa desde el diseño.